

```

In [ ]: #Data Structures ---Collection of data types
# In-built data structure
# 1. List
# 2. Tuple
# 3. Set
# 4. dictionary
# 5. range
#-----
#User define data structure
# 1. Stack
# 2. Queue
# 3. Tree
# 4. LinkedList
# We mostly focus on inbuilt datastructure
# User data structure take care by library

# 3- matrix -- collection of vector |

# collection of datastructure

# library -- NUMPY

# 4- dataframe - table - collection of rows & columns

# libray -- pandas

# 5- tensor -- collection of matrix

# library or package - pytorch == python + torch

# DATATYPE vs DATASTRUCTURE vs MATRIX vs PANDAS vs Tensor

# 1. The Variable (Single Value)

# [ Scalar / Variable ]

# [  ]
# [ 5 ]
# [  ]

# —> a = 5 —> Holds one single value in memory.

# _____

# 2. Vector (1D Array)

# 0 1 2 3

# [ ]
# [ 5 | 6 | 7 | 8 ]
# [ ]

# —> b = [5, 6, 7, 8] —> A linear collection with one dimension (length).

```

```

# -----

# 3. Matrix (2D Array)

# — COLUMNS —>

# 0  1  2

# | | | | |
# | 0 | 0 | 1 | 2 | ▲
# | | | | | |
# | 1 | 3 | 4 | 5 | ROWS
# | | | | | |
# | 2 | 6 | 7 | 8 | ▼
# | | | | |

# —> c = [[0, 1, 2], [3, 4, 5], [6, 7, 8]] —> [ NUMPY ]

# A grid of rows and columns, usually same data type.

# -----

# 4. Dataframe (2D Table with Labels)

# 1      2      3      4

# NAME    AGE    CITY    NAME

# | | | | |
# | 0 | Alice | 28 | NY | Derna |
# | | | | |
# | 1 | Aka  | 28 | 13 | Derna |
# | | | | |
# | 2 | Brath | 25 | US | Derna |
# | | | | |
# | 3 | Diveese | 29 | NY | Derna |
# | | | | |

# —> d = table —> [ PANDAS ]

# A table structure with Labeled rows and columns for mixed data types.

# -----

# 5. Tensor (3D Array)

```

```
# / / / |  
# / / / | Height  
# / / / ▼  
# ┌───┬───┬───┐  
# | 0 | 1 | 2 | ▲  
# ┌───┬───┬───┐ |  
# | 3 | 4 | 5 | Depth / Matrices stacked behind each other  
# ┌───┬───┬───┐ |  
# | 6 | 7 | 8 | ▼  
# ┌───┬───┬───┐  
# ← Width →  
# —→ e = multiple blocks —→ [ PYTORCH ]  
# Matrices stacked in multiple dimensions (3D+), forming deep layers.  
# (dt) —→ [ 1D List Block ]  
# / |  
# / ▼  
# (a) —→ 5 ─────────── (d) —→ [ 2D Table Grid ]  
# (table / dataframe)  
# (b) —→ [ 5, 6, 7, 8 ] (1D Vector Array)  
# |  
# └──→ Combines into... ┘  
# ▼  
# (c) —→ [ 0 1 2 ] (e) —→ [ [Matrix 1] [Matrix 2] ]  
# [ 3 4 5 ] [ [Matrix 3] [Matrix 4] ]  
# [ 6 7 8 ] (3D Tensor Blocks /  
# (2D Matrix) collection of matrix)  
# _____  
  
# DETAILED STRUCTURAL DIAGRAMS:  
  
# # 1. The Variable / Scalar (a)  
  
# # Holds a single item in memory.
```

```

# # Your Board: a = 5

# ┌───┐
# │ 5 │
# └───┘

# # 2. Vector / 1D Array (b)

# # A single linear row or List.

# # Your Board: b = [5, 6, 7, 8]

# ┌───┴───┴───┴───┐
# │ 5 │ 6 │ 7 │ 8 │
# └───┴───┴───┴───┘

# # 3. Matrix / 2D Array (c)

# # Stacking rows to form a flat 2D grid with rows and columns.

# # Your Board: c = 0 to 8 matrix

# ┌───┴───┴───┐
# │ 0 │ 1 │ 2 │
# └───┴───┴───┘
# ┌───┴───┴───┐
# │ 3 │ 4 │ 5 │
# └───┴───┴───┘
# ┌───┴───┴───┐
# │ 6 │ 7 │ 8 │
# └───┴───┴───┘

# # 4. Dataframe / Table (d)

# # A spreadsheet structure with column headers and row indexes.

# # Your Board: d = table

# Col 1   Col 2   Col 3   Col 4

# ┌───┴───┴───┴───┴───┐
# │ 0 │   │   │   │   │
# └───┴───┴───┴───┴───┘
# ┌───┴───┴───┴───┴───┐
# │ 1 │   │   │   │   │
# └───┴───┴───┴───┴───┘
# ┌───┴───┴───┴───┴───┐
# │ 2 │   │   │   │   │
# └───┴───┴───┴───┴───┘

```

```
# ┌───┬───┬───┬───┐

# # 5. Tensor / 3D Block Array (e)

# # Multiple 2D matrices stacked behind each other like depth layers.

# # Your Board: e = collection of matrix

# ┌───┬───┬───┐
# /   /   /|
# ┌───┬───┬───┐ |
# /   /   /| |
# ┌───┬───┬───┐ | /
# |   |   | | |
# ┌───┬───┬───┐ | /
# |   |   | | /
# ┌───┬───┬───┐
```

```
In [4]: # List
l = []
l
```

```
Out[4]: []
```

```
In [5]: len(l)
```

```
Out[5]: 0
```

```
In [6]: l1 = [10,20,30]
l1
```

```
Out[6]: [10, 20, 30]
```

```
In [7]: len(l1)
```

```
Out[7]: 3
```

```
In [8]: l1.append(40)
l1.append(50)
l1.append(60)
l1.append(10)
```

```
In [9]: l1
```

```
Out[9]: [10, 20, 30, 40, 50, 60, 10]
```

```
In [10]: len(l1)
```

```
Out[10]: 7
```

```
In [ ]: #Interview : Explain list means then we can explain below points
# 1. List is defined in square brackets
# 2. List is growable
# 3. Duplicate is Allowed in List
# 4. Append()----> Will add the value at the Last
# 5. List Append takes only one argument
# 6. List inside List is called the nested list and this can be achieved by append
# 7. In List we can pass any data type
# In List multiple data is allowed
# Copy() ----It copy one List to other List
# clear()----- It will clear(delete) the elements in the List
# count() -----It will give no of occurrences of a value
```

```
In [11]: l1.append(50,60)
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[11], line 1
----> 1 l1.append(50,60)

TypeError: list.append() takes exactly one argument (2 given)
```

```
In [12]: # Nested List
l1.append(['a','b','c'])
l1
```

```
Out[12]: [10, 20, 30, 40, 50, 60, 10, ['a', 'b', 'c']]
```

```
In [13]: # Inside list we can pass mixed data type
l2 = [34,5.3,'Bheemraj',1+2j,True]
l2
```

```
Out[13]: [34, 5.3, 'Bheemraj', (1+2j), True]
```

```
In [14]: print(l)
print(l1)
print(l2)

[]
[10, 20, 30, 40, 50, 60, 10, ['a', 'b', 'c']]
[34, 5.3, 'Bheemraj', (1+2j), True]
```

```
In [15]: print(l)
print(l1)
print(l2)

[]
[10, 20, 30, 40, 50, 60, 10, ['a', 'b', 'c']]
[34, 5.3, 'Bheemraj', (1+2j), True]
```

```
In [16]: print(id(l))
print(id(l1))
print(id(l2))

2595294735808
2595294735104
2595298864128
```

```
In [17]: print(type(l))
print(type(l1))
```

```
print(type(l2))
```

```
<class 'list'>  
<class 'list'>  
<class 'list'>
```

```
In [18]: # List membership  
10 in l1
```

```
Out[18]: True
```

```
In [19]: ['a', 'b', 'c'] in l1
```

```
Out[19]: True
```

```
In [20]: 100 in l1
```

```
Out[20]: False
```

```
In [21]: l3 = l2.copy()
```

```
In [22]: l3
```

```
Out[22]: [34, 5.3, 'Bheemraj', (1+2j), True]
```

```
In [23]: print(type(l))  
print(type(l1))  
print(type(l2))  
print(type(l3))
```

```
<class 'list'>  
<class 'list'>  
<class 'list'>  
<class 'list'>
```

```
In [24]: print(l)  
print(l1)  
print(l2)  
print(l3)
```

```
[]  
[10, 20, 30, 40, 50, 60, 10, ['a', 'b', 'c']]  
[34, 5.3, 'Bheemraj', (1+2j), True]  
[34, 5.3, 'Bheemraj', (1+2j), True]
```

```
In [26]: l3.clear()
```

```
In [27]: print(l3)  
print(len(l3))
```

```
[]  
0
```

```
In [28]: # If we want to delete complete l3 from memory then we need to use the del keywo  
del l3
```

```
In [29]: print(l3)
```

```
-----  
NameError                                Traceback (most recent call last)  
Cell In[29], line 1  
----> 1 print(13)  
  
NameError: name '13' is not defined
```

```
In [31]: 12.count(34)
```

```
Out[31]: 1
```

```
In [32]: 11[:]
```

```
Out[32]: [10, 20, 30, 40, 50, 60, 10, ['a', 'b', 'c']]
```

```
In [33]: 11[-1]
```

```
Out[33]: ['a', 'b', 'c']
```

```
In [34]: 11[100]
```

```
-----  
IndexError                                Traceback (most recent call last)  
Cell In[34], line 1  
----> 1 11[100]  
  
IndexError: list index out of range
```

```
In [35]: 11[4:]
```

```
Out[35]: [50, 60, 10, ['a', 'b', 'c']]
```

```
In [37]: 11[3:-1]
```

```
Out[37]: [40, 50, 60, 10]
```

```
In [38]: 11
```

```
Out[38]: [10, 20, 30, 40, 50, 60, 10, ['a', 'b', 'c']]
```

```
In [39]: 11[0:10:3]
```

```
Out[39]: [10, 40, 10]
```

```
In [40]: 11[::1]
```

```
Out[40]: [10, 20, 30, 40, 50, 60, 10, ['a', 'b', 'c']]
```

```
In [41]: 11[::2]
```

```
Out[41]: [10, 30, 50, 10]
```

```
In [42]: 11[::3]
```

```
Out[42]: [10, 40, 10]
```



```
In [43]: # Reversing All the Elements  
11[::-1]
```

```
Out[43]: [['a', 'b', 'c'], 10, 60, 50, 40, 30, 20, 10]
```

```
In [44]: # Reversing -2 Elements  
11[::-2]
```

```
Out[44]: [['a', 'b', 'c'], 60, 40, 20]
```

```
In [45]: 11[::-3]
```

```
Out[45]: [['a', 'b', 'c'], 50, 20]
```

```
In [ ]:
```